MODULE

# 10

# Debugging with history

History is evidence: filter the log, blame a line, search with the pickaxe, and bisect a bug.

YOU WILL LEARN

| | |
|---|---|
| git log like a detective | 01_log_like_a_pro.md |
| blame & the pickaxe | 02_blame_and_pickaxe.md |
| git bisect | 03_bisect.md |
| Exercise: find the bug | ex10_find_the_bug/ |
| Interview Q&A | INTERVIEW.md |

WHY DEBUGGING

# ”It worked last week”

## THE PROBLEM

Prices are wrong in production. 200 commits went in since the last good release. Reading them one by one takes a day, and you don’t even know which file to look at.

## THE IDEA

**Git history is evidence. Filter it (log), trace a line (blame, -S), and let bisect find the culprit in ~8 steps.**

TRY IT GIT LOG

# Filter the history

```
$ git log --oneline --author=Priya
4034e86 Coupon: expire after 30 days
3629b31 Add coupon codes
$ git log --oneline -i --grep=coupon
4034e86 Coupon: expire after 30 days
e0d95c2 Document coupons
3629b31 Add coupon codes
$ git log --oneline -- cart.py
766dba7 Fix rounding in cart
e426c38 Add cart
$ git shortlog -sn HEAD
     3  Vishal
     2  Priya
     1  Raj
```

1. By person, by message, by file.
2. Also **--since**, **--until**, **--follow**, **--no-merges**.
3. **shortlog**: commits per author.

TRY IT BLAME AND PICKAXE

# Who changed this line? When did it appear?

```
$ git blame -s -L 1,3 cart.py
c2a7a6f3 1) def total(items, discount=0):
b863aded 2)     rate = 1 - min(discount, 0.5)
c2a7a6f3 3)     return sum(items) * rate
$ git log --oneline -S discount
c2a7a6f Add discount
$ git log --oneline -S "min("
b863ade Cap discount at 50%
$ git grep -n rate HEAD~1
HEAD~1:cart.py:2:    rate = 1 - discount
HEAD~1:cart.py:3:    return sum(items) * rate
```

1. **blame**: the last commit per line.
2. **-S** (pickaxe): commits that added/removed the text.
3. **git grep** searches any version.

PICTURE IT BISECT

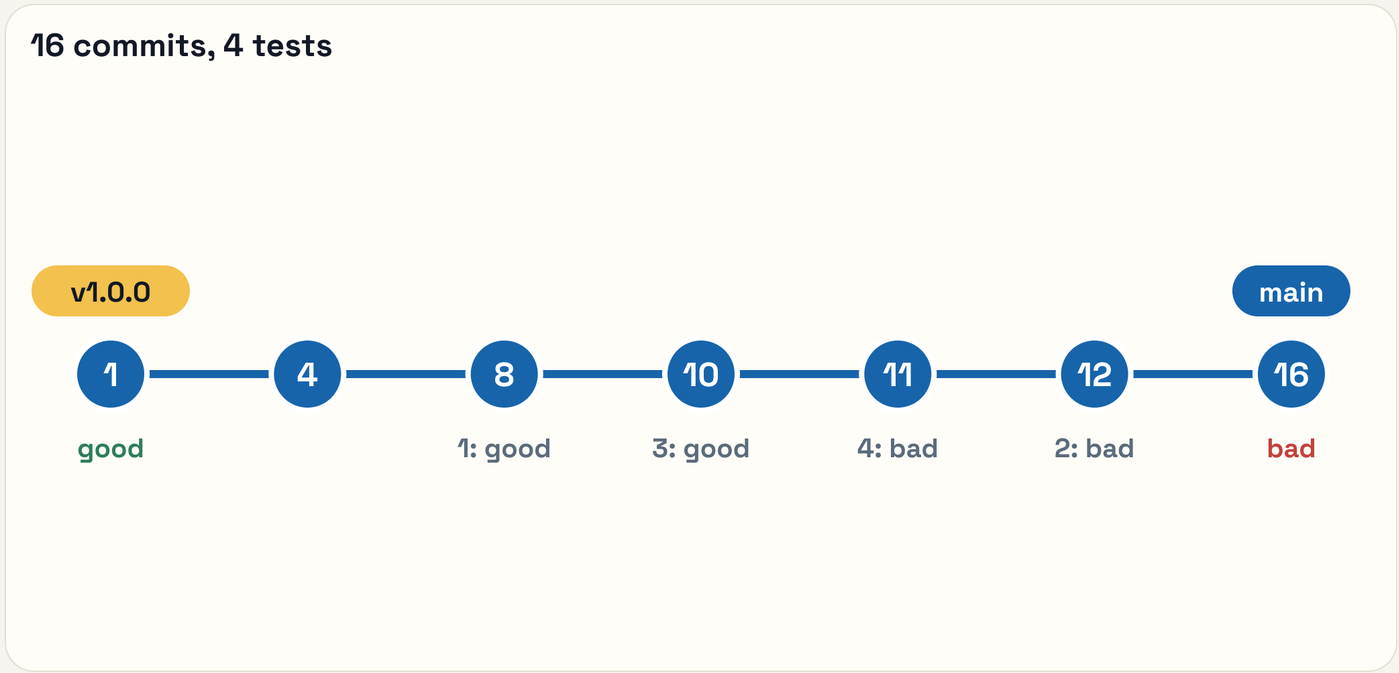
# bisect = the number-guessing game

🎯 ”I’m thinking of a number between 1 and 1,000.” ”Higher or lower?” Halving each time finds it in 10 guesses. bisect does the same with commits: **good or bad?**



PICTURE IT BISECT

# Binary search through history




1. Test the middle, throw away half.
2. 16 commits → 4 tests; 1,000 → ~10.
3. **11** is the first bad commit.

TRY IT BISECT

# Let a script do the testing

```
$ git bisect start HEAD v1.0.0
Bisecting: 7 revisions left to test after this (roughly 3 steps)
[3de9aec6c0e8012786e5cd0044dc2b1e0d22e570] Change 8
$ git bisect run sh check_price.sh
running 'sh' 'check_price.sh'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[a88ec4c0211c576eb0e32236fa3f1b5e6b03ad25] Change 12
running 'sh' 'check_price.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[2059072525fea07f539dcb4609ab43ffb04b776e] Change 10
running 'sh' 'check_price.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[ccd5ec2d091dd68eec1516a4d09f2cf0704d5a80] Change 11
running 'sh' 'check_price.sh'
ccd5ec2d091dd68eec1516a4d09f2cf0704d5a80 is the first 'bad' commit
commit ccd5ec2d091dd68eec1516a4d09f2cf0704d5a80
Author: Vishal <vishal@example.com>
Date:   Tue Oct 27 08:23:00 2026 +0000

    Change 11

 CHANGELOG.txt | 1 +
 price.txt     | 2 +-
 2 files changed, 2 insertions(+), 1 deletion(-)
bisect found first 'bad' commit
```

1. Exit code 0 = good, 1-127 = bad, 125 = skip.
2. 16 commits → 4 checks.
3. Then **git bisect reset** to go back to main.

STEP BY STEP DEBUGGING

# Which tool for which question?

| QUESTION | TOOL |
| --- | --- |
| What changed recently in payments/? | `git log --oneline --since="1 week ago" -- payments/` |
| Who wrote this odd line, and why? | `git blame -L 40,60 file` → `git show <id>` → the PR |
| When was this function removed? | `git log -S "def apply_coupon" --oneline` |
| Which commit broke the build? | `git bisect run pytest -x` |
| Is this TODO in the released version? | `git grep TODO v2.3.0` |

COMMON MISTAKE BISECT

# A flaky test leads bisect astray

✗ WRONG

```
git bisect run pytest
# runs the WHOLE suite: unrelated
# flaky tests fail randomly
```

bisect trusts every answer. One wrong "bad" sends it into the wrong half, and it confidently blames an innocent commit.

✓ RIGHT

```
git bisect run pytest tests/test_prices.py -x
# a small, reliable test that
# reproduces exactly this bug
```

Write a tiny, deterministic reproduction script first, then bisect with it.

IN REAL JOBS DEBUGGING

# History at work

### Production regression

```
git bisect start HEAD v2.3.0
git bisect run ./repro.sh
```

### Understand a decision

```
git blame -L 120,130 billing.py
git show a1b2c3d   # + the linked PR
```

### Ignore formatting commits in blame

```
echo <reformat-id> >> .git-blame-ignore-revs
```

### Release-level history

```
git log --first-parent --oneline main
```

INTERVIEW SAY THIS IN THE INTERVIEW

# “How do you find which commit introduced a bug?”

- **git bisect** between a known good commit (last release tag) and a bad one (HEAD).
- Binary search: about **log2(n)** steps, so 1,000 commits need ~10 checks.
- Best with **git bisect run** and a small script that reproduces the bug.
- Then **git show** the culprit, read its PR, write a test, fix.
- For a single line: **git blame**; for added/removed code: **git log -S**.

> **Follow-up they ask next:** ”A commit in the range doesn’t build?” (git bisect skip, or exit 125 from the script).

RECAP REMEMBER THESE

# Module 10 in 5 lines

1. **log** filters: --author, --grep, --since, -- path, --follow.
2. **blame -L**: who last changed these lines.
3. **log -S / -G**: when code appeared or disappeared.
4. **git grep**: search any version.
5. **bisect run**: the first bad commit in ~log2(n) steps.

PRACTICE NOW

**Do exercises/ex10_find_the_bug/: 30 commits, one bug, bisect it.**